Qt 使用 GDB 调试



正点原子公司名称 ：广州市星翼电子科技有限公司

原子哥在线教学平台：www.yuanzige.com

开源电子网 / 论坛 ：http://www.openedv.com/forum.php

正点原子淘宝店铺 ：https://openedv.taobao.com

正点原子官方网站 ：www.alientek.com

正点原子 B 站视频 ：https://space.bilibili.com/394620890

电话：020-38271790 传真：020-36773971

请关注正点原子公众号，资料发布更新我们会通知。



扫码关注正点原子公众号

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	初稿：	正点原子	正点原子	2025.07.07

目录

前言 5
第一章 Buildroot 中配置 GDB.....8
1.1 Qt 配置 GDB 调试注意事项 9
1.2 在 Buildroot 中添加 GDB 调试配置 9
第二章 Qt 使用 GDB 调试.....11
2.1 搭建 Qt 远程部署环境.....12
第三章 Qt 使用 GDB.....15
3.1 Qt 使用 GDB 调试 16

前言

为了响应正点原子用户需要，本文档为 Qt 配置 GDB 调试文档。默认基于 Buildroot 的出厂系统都不支持 Qt 配置 GDB，因为发布的镜像为“Release”版本，而 Qt 配置 GDB，需要将系统编译成“Debug”版本，存在大量的调试信息，可能会影响系统运行！

本文属于原创教程，转载请附带正点原子出处，请勿商用！

原创出处，在几年前笔者在 RV1126 用户群里就回复我们用户，如今将它写成文档。



在正点原子所有 Buildroot 构建的系统里，都是没有配置 Qt GDB 调试的，但是 Yocto 构建的系统 IMX6ULL、STM32MP157 等都可以在 Qt 里都是可以使用 GDB 的，配置也是非常简单。并且笔者认为在 Qt 里配置 GDB 调试不是必须的。一些简单的程序没有必要使用 GDB 调试，一般都是用 qDebug()打印即可，因为如果是个人写的程序的话，在编写程序时就已经注意到内存溢出，程序的健壮问题。如果是大项目，或者接手的项目，有需要调试，就可能需要 GDB 调试。

1. 深度集成 Qt 框架

Qt 对象调试支持：GDB 通过扩展脚本（如 qt5.py）可直接显示 Qt 对象的属性（如 QObject 的子对象、信号槽连接等），无需手动展开内存结构。

元对象系统（MOC）支持：能解析 Qt 的元信息（如 Q_OBJECT 生成的代码），方便调试信号槽、属性系统等。

QString/QVariant 友好显示：调试时直接显示 QString 的文本内容（而非 UTF-16 指针）和 QVariant 的实际类型值。

2. 强大的多线程调试能力

线程列表与切换：GDB 可列出所有线程，并允许随时切换上下文查看不同线程的调用栈和变量状态。

线程同步问题定位：结合断点条件（如 break if thread_id == X）和锁等待检测，快速定位死锁或竞态条件。

3. 灵活的断点与条件调试

条件断点：支持复杂条件（如 break main.cpp:100 if x > 10），避免频繁手动暂停。

观察点（Watchpoint）：监控变量被修改的时机，适合调试意外修改的数据。

临时断点：通过 tbreak 命令设置一次性断点，减少干扰。

4. 内存与底层问题排查

指针与内存分析：直接查看内存地址内容（如 x/10xb 0x1234），适合调试内存泄漏、越界访问等问题。

反汇编调试：结合 disassemble 命令分析崩溃时的汇编代码，定位底层错误。

5. 跨平台一致性

统一调试体验：GDB 在 Linux、macOS 和 Windows（通过 MinGW 或 Cygwin）上行为一致，减少平台适配成本。

与 Qt Creator 无缝协作：Qt Creator 内置 GDB 前端，提供图形化界面（如变量监视、调用栈导航），同时保留 GDB 命令行的高级功能。

6. 脚本与自动化调试

GDB 脚本：编写 .gdb 脚本自动化重复调试任务（如批量设置断点、记录日志）。

Python 扩展：利用 GDB 的 Python API 开发自定义调试命令，例如统计信号槽调用次数。

7. 性能分析辅助

采样调试：通过 monitor 命令结合 perf 或 valgrind，分析 CPU 使用率或内存分配热点。

时间旅行调试：结合 rr（Record & Replay）工具，回放程序执行过程，精准定位偶发问题。

8. 与 Qt 工具链协同

结合 qmlscene 调试 QML：通过 GDB 附加到 QML 引擎进程，调试 JavaScript 逻辑与 C++ 交互。

调试 Qt 插件：动态加载的插件可通过 GDB 符号表分析加载失败原因。

适用场景示例

崩溃调试：通过核心转储（core dump）或 catch throw 捕获异常，结合调用栈定位崩溃点。

信号槽问题：检查 QObject::connect 是否成功，或信号是否被意外断开。

渲染问题：调试 Qt Quick 场景图（Scene Graph）的 GPU 数据传递错误。

对比其他工具

LLDB：macOS 默认调试器，性能更优，但 Qt 支持不如 GDB 完善。

Visual Studio Debugger：Windows 平台更友好，但跨平台能力弱。

Qt Creator 内置调试器：适合快速操作，但复杂场景仍需直接使用 GDB 命令。

以上来自 AI 总结，好了，现在我们来快速配置 GDB 吧。

免责声明

本文档所提及的产品规格和使用说明仅供参考，如有内容更新，恕不另行通知；除非有特殊约定，本文档仅作为产品指导，所作陈述均不构成任何形式的担保。本文档版权归广州市星翼电子科技有限公司所有，未经公司的书面许可，任何单位和个人不得以营利为目的进行任何方式的传播。

为了得到最新版本的产品信息，请用户定时访问正点原子资料下载中心或者与淘宝正点原子旗舰店客服联系索取。感谢您的包容与支持。

第一章 Buildroot 中配置 GDB

本章为 Buildroot 中配置 GDB。在正点原子提供的 Buildroot 中，有些开发平台是没有配置 GDB 的，因为某些开发平台存储容量紧张，如正点原子 RK3506，有 256MB NANDFLASH 的配置。存储资源还是比较紧张的，所以默认是没有配置。

1.1 Qt 配置 GDB 调试注意事项

有些开发板平台默认已经编译 GDB，如正点原子 RK3568、RK3588 等。但是配置了 GDB，Qt 不能直接使用 GDB 调试，因为 Qt 还需要将调试信息编译。所以完整的配置如下。添加以下配置到 Buildroot 的 defconfig 文件里。

```
#工具链 GDB
BR2_PACKAGE_HOST_GDB=y
BR2_PACKAGE_HOST_GDB_TUI=y
BR2_PACKAGE_HOST_GDB_PYTHON3=y
BR2_PACKAGE_HOST_GDB_SIM=y

#目标主机（开发板）
BR2_PACKAGE_GDB=y
BR2_PACKAGE_GDB_SERVER=y
BR2_PACKAGE_GDB_DEBUGGER=y
BR2_PACKAGE_GDB_TUI=y

#重点开启下面两项，运行时 Debug 和编译 Debug 信息。
#请注意，此两项，在调试阶段可选上编译，但是生产阶段，建议不要开启此两项。
#此两项可能会增加二进制程序体积，并且可能影响程序运行效率！
BR2_ENABLE_RUNTIME_DEBUG=y
BR2_ENABLE_DEBUG=y
```

1.2 在 Buildroot 中添加 GDB 调试配置

以 ATK-DLRK3506 为例。在<SDK>/buildroot/configs/atk_dlrk3506_defconfig 文件中添加 1.1 小节的配置，如下图。



由于添加了全局配置，会影响整个系统，所以我们需要清除 Buildroot 再编译！

执行下面指令清除 Buildroot。

```
./build.sh clean-rootfs
```

```
alientek@alientek:~/ATK-DLRK3506$ ./build.sh clean-rootfs
```

然后再重新编译 Buildroot。

```
./build.sh rootfs      # 或者直接执行./build.sh
```

```
alientek@alientek:~/ATK-DLRK3506$ ./build.sh rootfs
```

编译生成的固件，再烧写到开发板即可。

第二章 Qt 使用 GDB 调试

本章以 ATK-DLRK3506 开发板为例，Qt 配置 GDB 调试。

2.1 搭建 Qt 远程部署环境

请参考网盘资料中的 10、用户手册下/xx/【正点原子】基于 Buildroot 系统_Qt 开发环境搭建参考手册，先搭建远程部署环境，必须先熟读和熟练配置 Kits，再往下做！笔者不会重复写步骤。

请先烧录 1.2 小节生成的含 GDB 配置的镜像，然后我们来配置 Qt GDB。

以 ATK-DLRK3506 为例，工具链一般在编译 Buildroot 时候生成，位于<SDK>/output/<目标平台> /host 路径下，如下图。

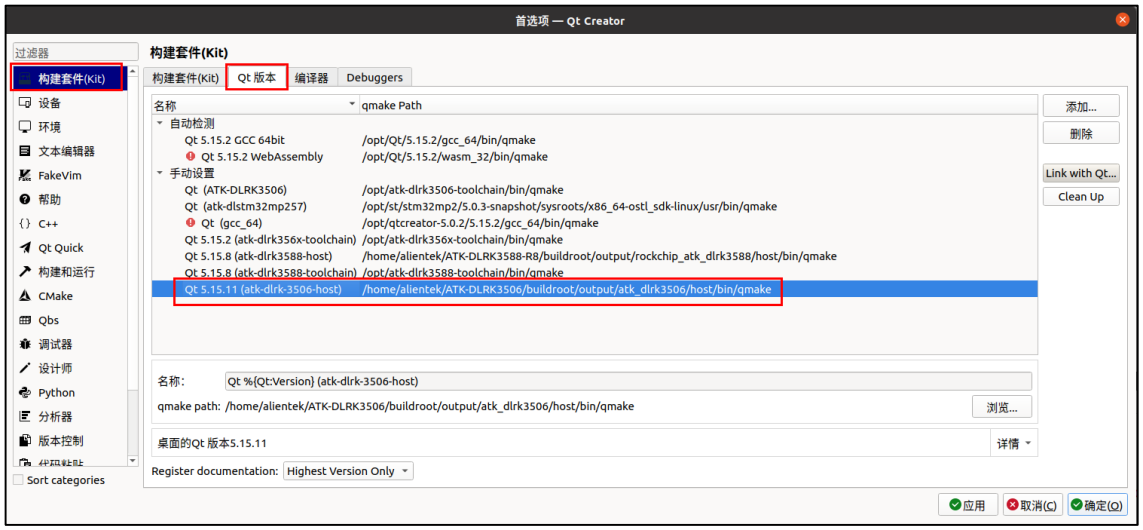
新的工具链路径/home/alientek/ATK-DLRK3506/buildroot/output/atk_dlrk3506/host。

```
alientek@alientek:~/ATK-DLRK3506/buildroot/output/atk_dlrk3506/host$ ls
arm-buildroot-linux-gnueabihf  bin  doc  environment-setup  etc  include  lib  lib64  libexec  man  mkspecs  sbin  share  usr
alientek@alientek:~/ATK-DLRK3506/buildroot/output/atk_dlrk3506/host$
```

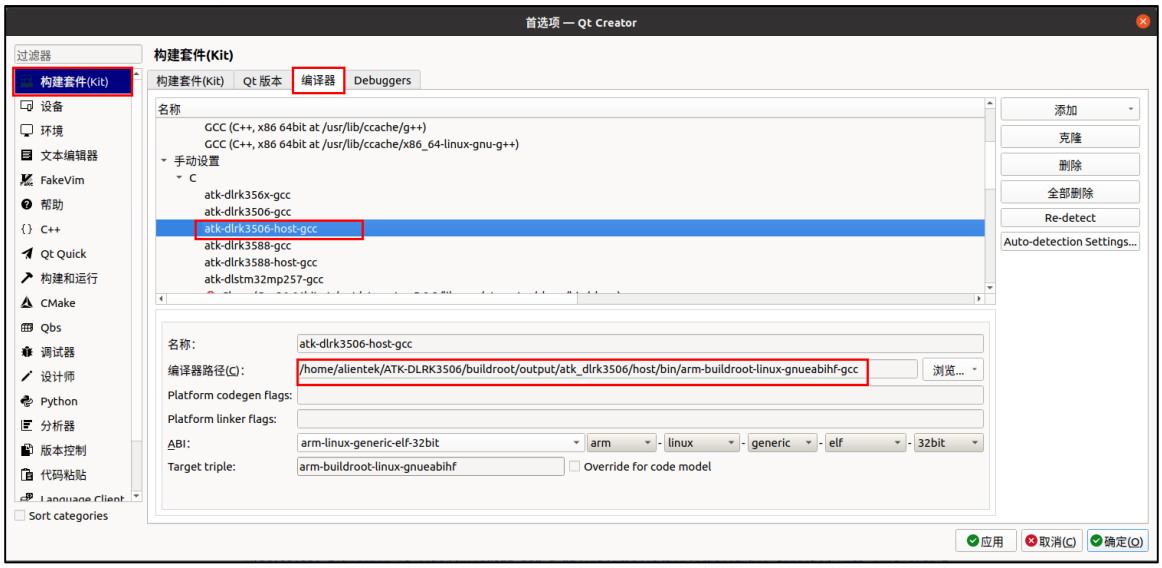
工具链里有 host/bin/arm-buildroot-linux-gnueabihf-gdb，是 GDB 的客户端。gdbserver 在 Buildroot 生成的根文件系统中，也就是开发板跑的系统，运行在 ARM 设备上，负责执行程序并响应 GDB 客户端的调试请求。

所以我们需要重新配置 Qt Kits（编译套件），让编译套件指向我们生成的工具链。如下图，笔者已经修改 Qt Kits 指向生成的工具链了。

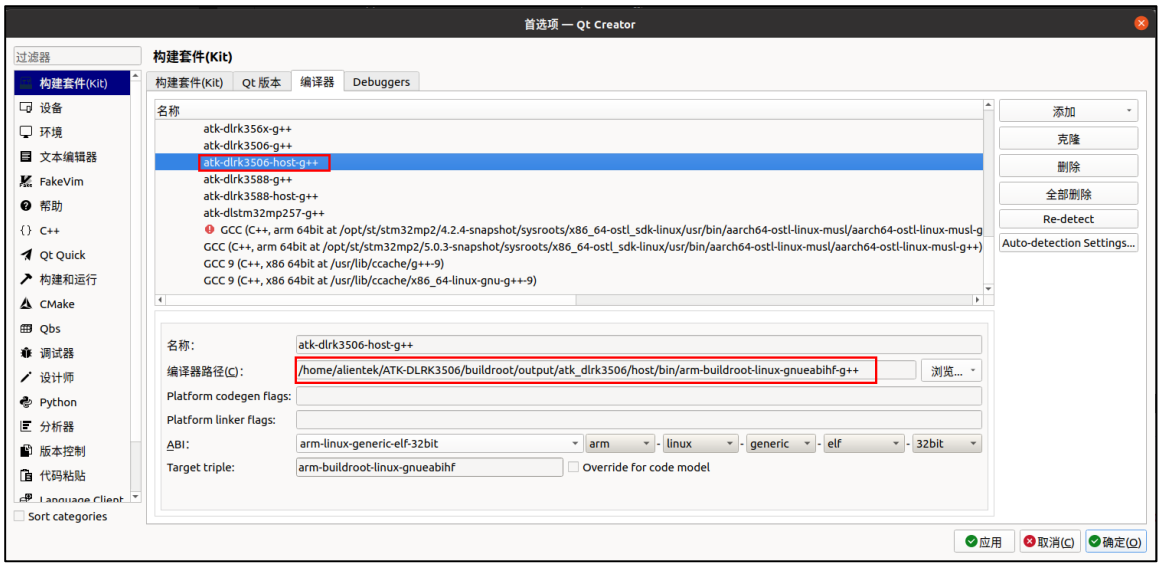
Qt 版本重新指向生成的工具链。



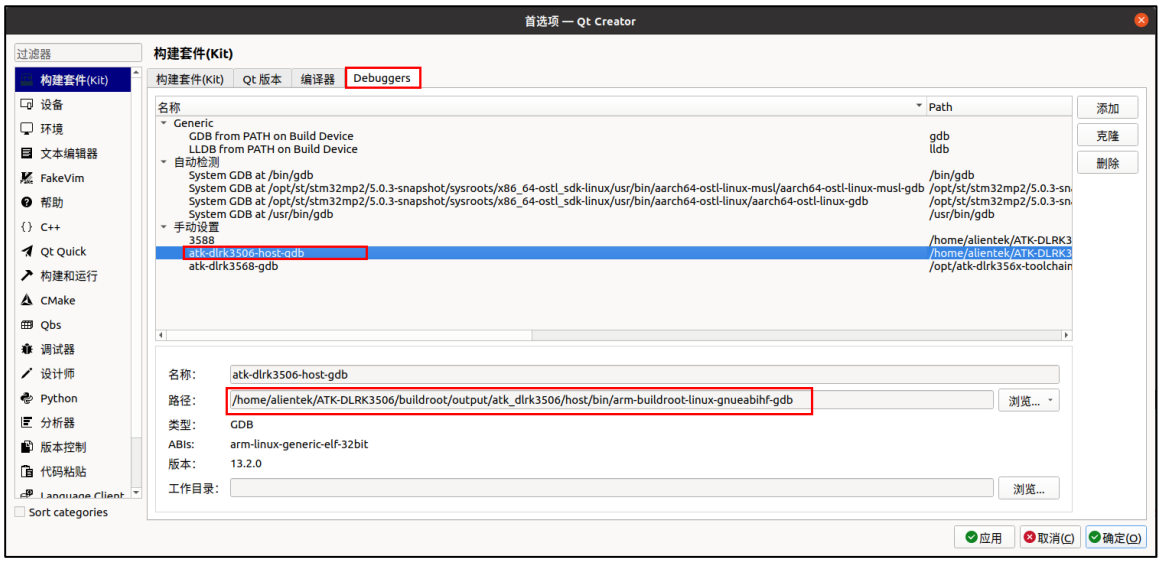
GCC 编译器重新指向生成的工具链。



G++编译器重新指向生成的工具链。



现在重点来了，配置 GDB。配置选择 Debuggers，然后命名为“atk-dlrk3506-host-gdb”。配置路径如下图所示。



回到 Kit 配置里，指定 GDB 调试，如下图。



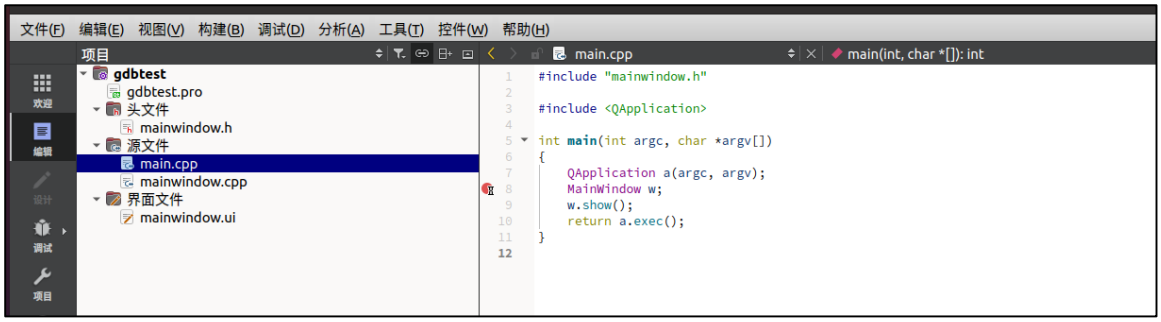
第三章 Qt 使用 GDB

本章来实战 Qt 使用 GDB 调试。

3.1 Qt 使用 GDB 调试

新建一个简单项目测试 GDB 调试，请勿使用大项目，防止用户的大项目自身存在许多编译错误，导致进行不下去！

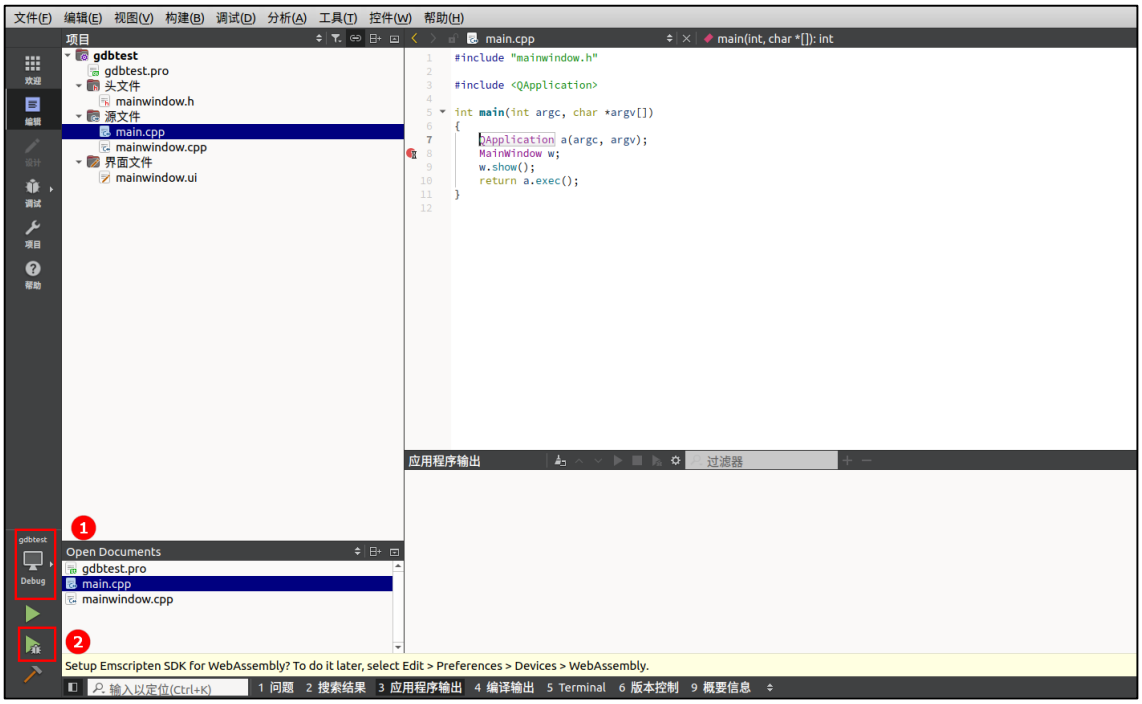
如下图，笔者创建了一个“gdbtest”项目，并且在第八行打上了断点。



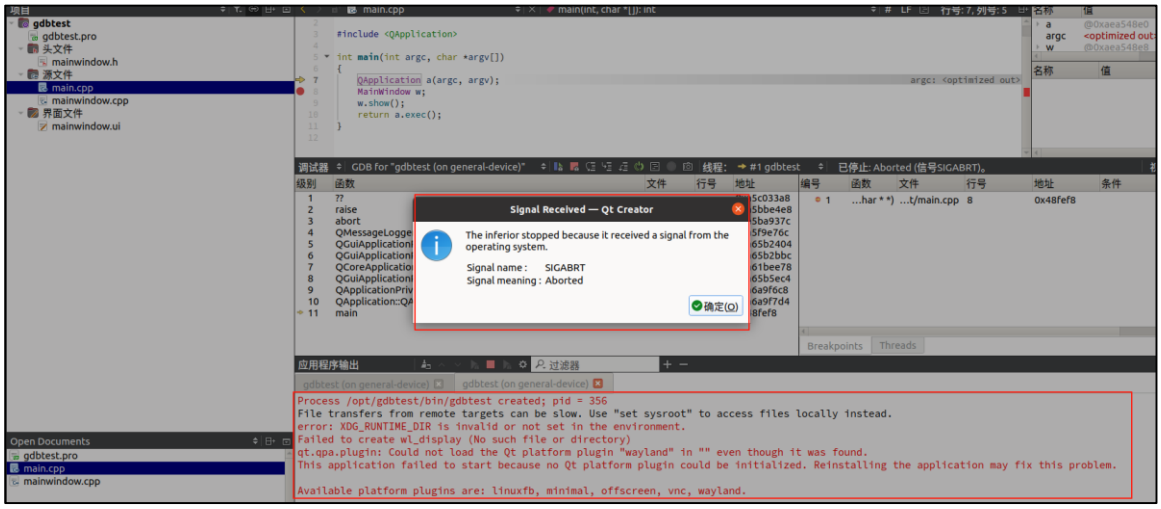
如果用户出现下图问题，原因是没有按 1.2 小节配置来编译 Buildroot！



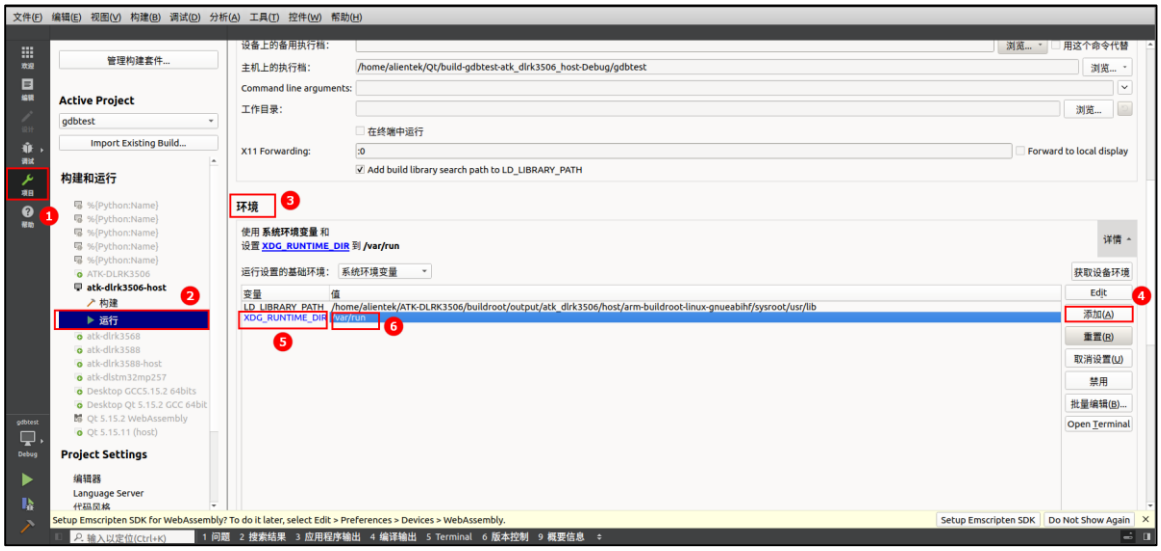
好了，现在我们来开始调试，首先确认选择 Debug 版本，然后开始 Debug 调试。



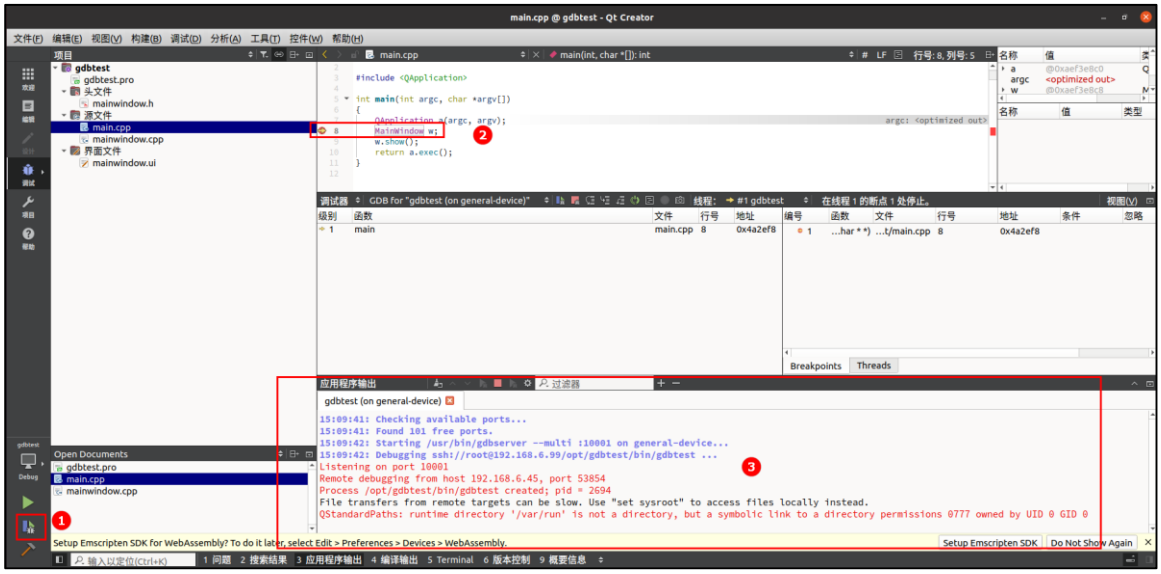
毫无意外，用户出现下图问题，原因是我们没有配置某个环境变量，因为在 RK 系统开发板中如 RK3506 RK3568 RK3588 等都是使用 Weston 显示 Qt 的。



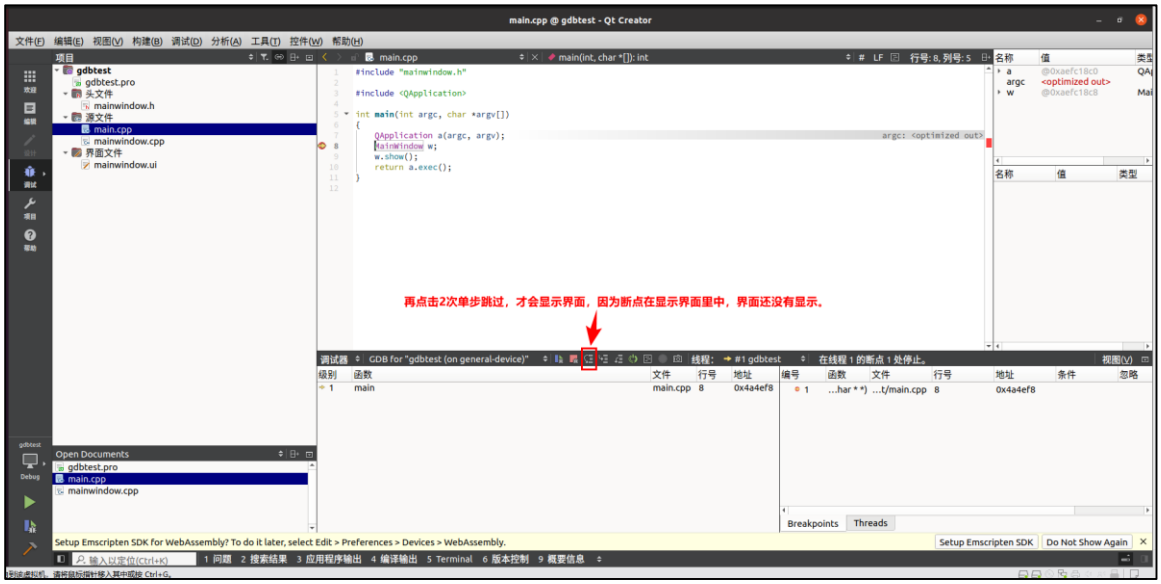
现在我们来处理这个问题，将 XDG_RUNTIME_DIR 环境变量写到运行环境变量中。找项目》运行》环境。将 XDG_RUNTIME_DIR 赋值为/var/run 即可。



然后关掉运行的项目，再重新 Debug。如下图，程序已经运行到断点处了。说明 GDB 已经正常运行了！



点击单步跳过，程序能进一行下一句代码，也说明 GDB 能正常调试。



至此 Qt GDB 调试教程已经结束。